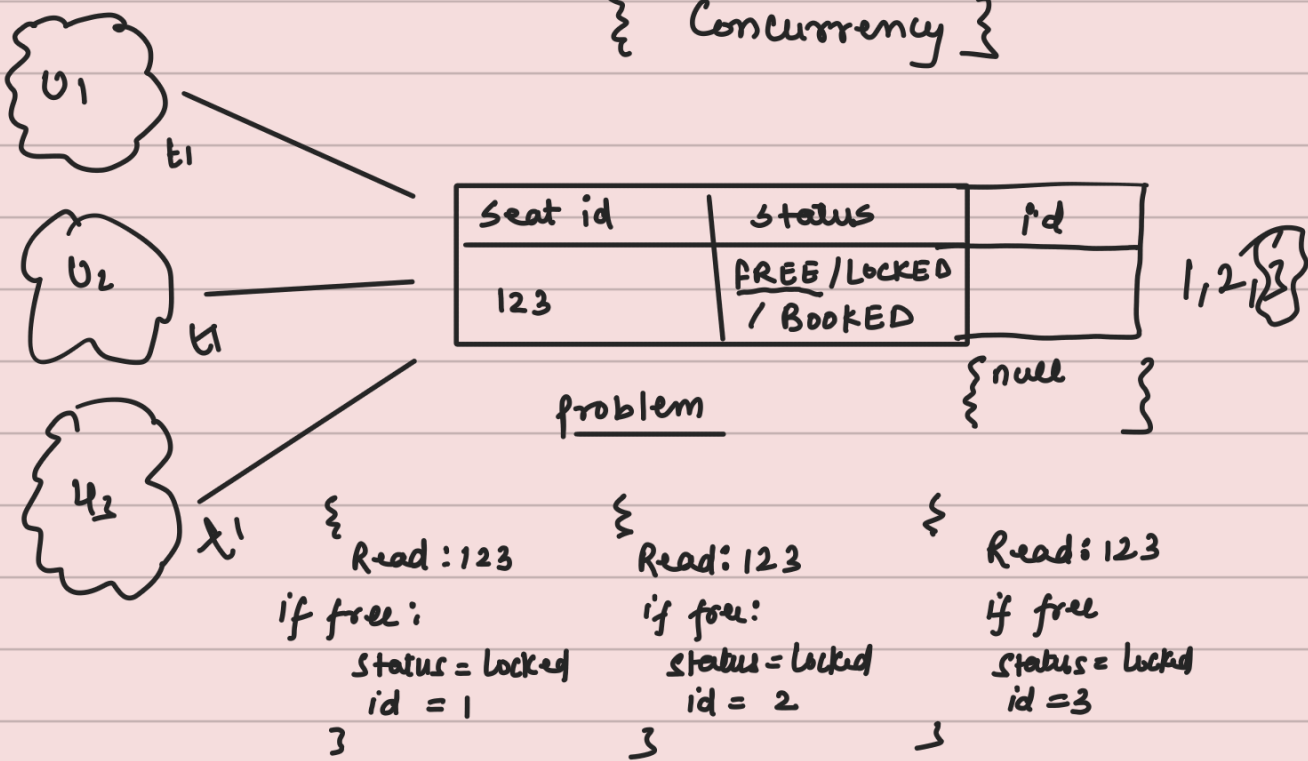


{ Concurrency }



Solution 1

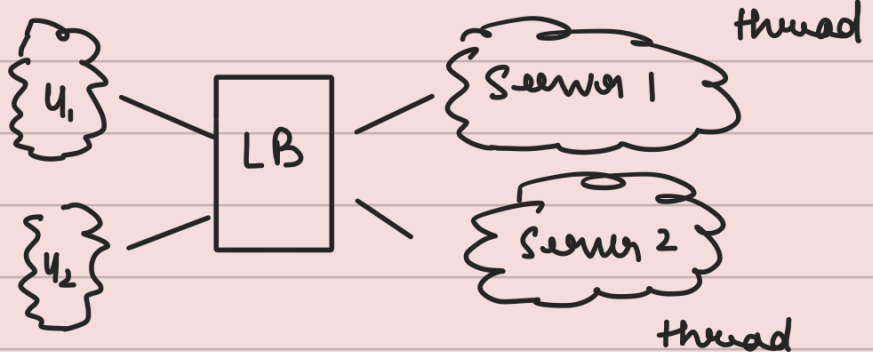
1) Single Thread { Synchronized - Java }

[Work Perfect
for 1 Server]

problem

1) Not work in Distributed

U1	U2	U3
✓	X	X



2). Use Distributed Concurrency Control

for Understanding

1) transaction?

Begin_txn:

- Debit money from x
- Credit money to y
- if all success:
Commit
- else:
Rollback;

initial = 10 + 0 = {10} ✓
 after State = 5 + 5 = {10} ✓
 (X) after State = 5 + (0) = (5) X

W₁

X

W₁ = 10

W₁ = 10 - 5

W₁ = 5

W₂

y

W₂ = 0

W₂ = 0 + 5

W₂ = 5

= 7

(12) X

W₃

2) DB locking?

Lock Type	Another Shared lock	Another Exclusive lock
Have Shared lock	Yes	No
Have Exclusive lock	No	No

1) U₁ → S(1) → x - 10 ✓
 U₂ → S(1) → x - 10 ✓

Set	Value
10	10

2) U₁ → S(1) → x - 10 ✓
 U₂ → X(1) → {x - 20} X

3) U₁ → X(1) → {x - 10, x - 20} ✓
 U₂ → S(1) → no Reading X

4) U₁ → X(1) → {x - 10, x - 20} ✓
 U₂ → X(1) → {x - 10, x - 20} X

3) What is isolation?

Isolation level	Dirty Read	non-Repeatable Read	phantom Read
Read uncommit	Yes	Yes	Yes
Read commit	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

1) Dirty Read

Begin_txn:

	Txn 1	Txn 2
T ₁	Read id=1, status = free	Read id=1, status = free
T ₂		write id=1, status = Booked
T ₃	Read id=1, status = Booked	
T ₄	{ Lol! Dirty Read }	Commit → failure {free}
	End_txn:	

2) Nm - Repeatable Read

Begin - txn

txn 1	
T ₁	Read id = 1, status = free
T ₂	<u>Begin - txn 2</u> Read id = 1, status = free write id = 1, status = Booked Commit success
T ₃	
T ₄	
T ₅	
	Read id = 1, status = Booked

{ logically correct →
But value changed in flow }

3) Phantom Read

1) Search in Range

Begin - txn:

txn 1	
T ₁	id = 1, status = free id = 4, status = Booked { id > 0 and id < 6 }
T ₂	id = 1, status = free id = 5, status = Booked { id > 0 and id < 6 } id = 4, status = Booked

3) What is isolation?

Isolation level	Dirty Read	Non-Repeatable Read	Phantom Read
Read uncommit	Yes	Yes	Yes
Read commit	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Read UnCommitted :-

1) No lock Acquired {shared, Exclusive}

Read Committed :-

- 1) Read: Shared lock and Release as Read is done.
- 2) Write: Exclusive lock $\rightarrow$ { till complete txn }.

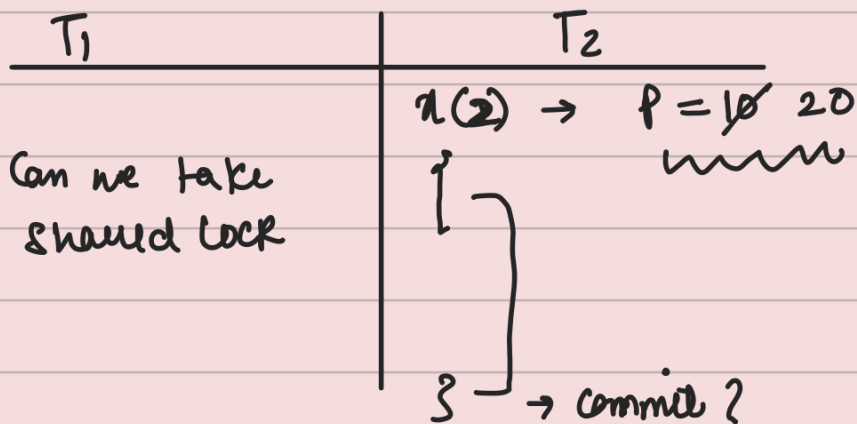
Repeatable Read

- 1) Read: Shared lock $\rightarrow$ { till end of txn }.
- 2) Write: Exclusive lock $\rightarrow$ { till end of txn }.

Serializable

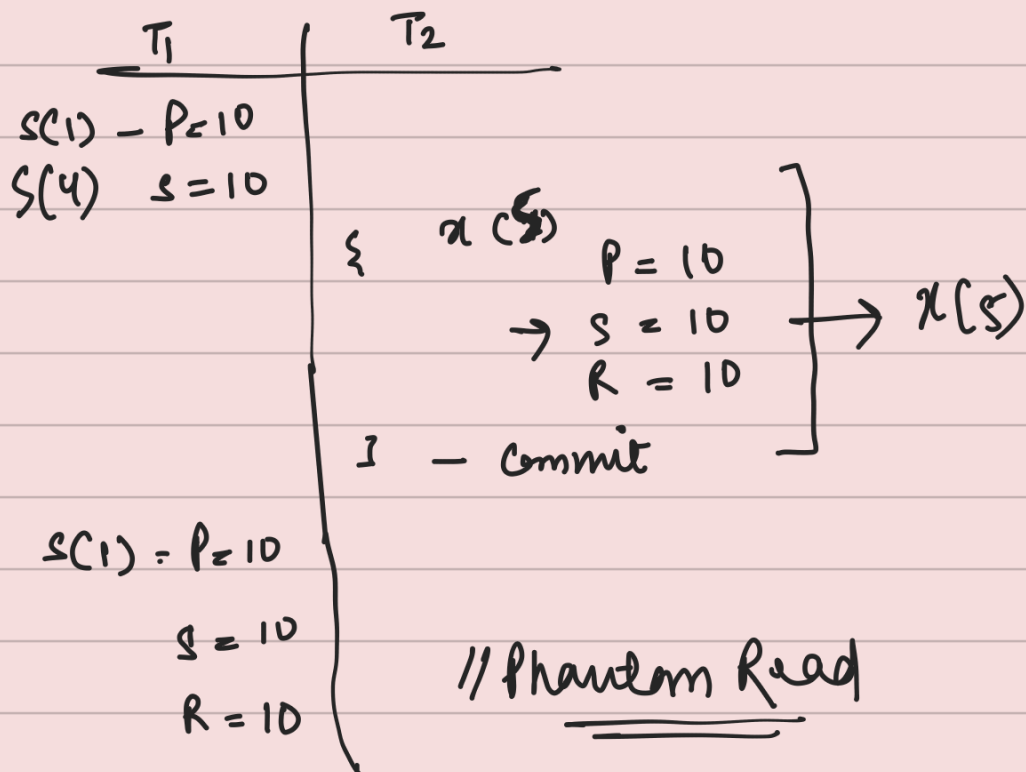
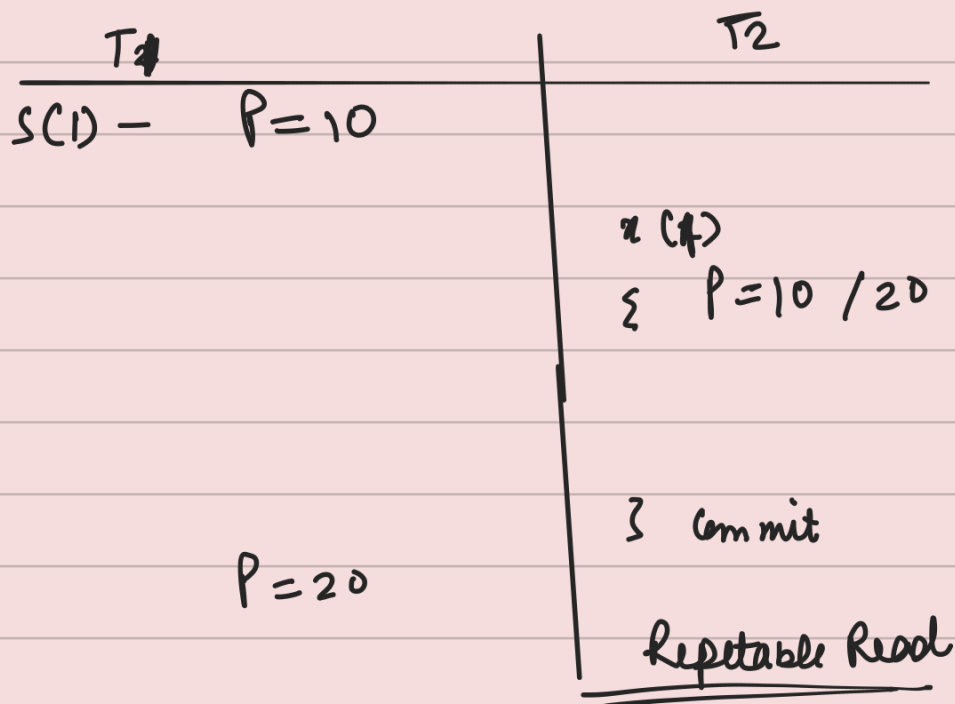
- 1) Repeatable Read
- 2) < Range lock >

Read Committed



S(1) or (x)(1)

→ no dirty read



Repetatable Read

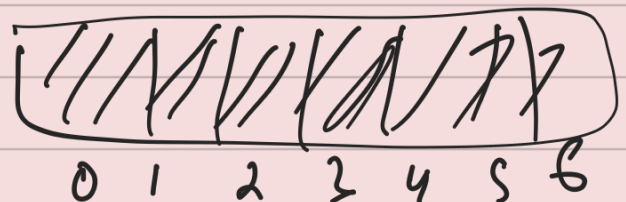
T_1	T_2
$S(1) \quad P = 10$	{ Can T_2 update it? no (x)
$S(1) \quad P = 10$	
	when T_1 is completed then you can take exclusive lock.

{ no Dirty Read
no Repetatable Read }

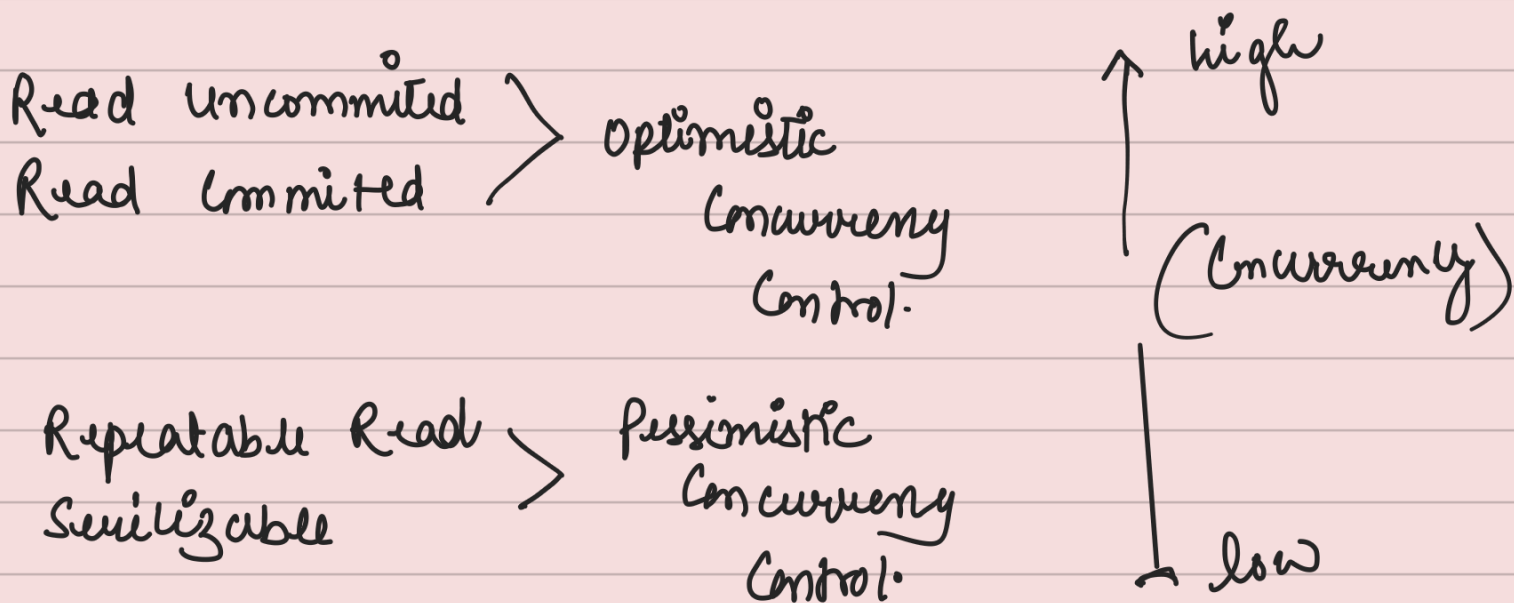
T_1 <Range>	T_2
$S(1) \quad P = 10$	$\alpha(3) = S = 20$
$S(4) \quad R = 10$	
$S(1) \quad P = 10$	// Phantom Read.
$S(4) \quad R = 10$	
$S(3) \quad S = 20$	

Serializable

T_1 (Range)	T_2
$S(1) - P = 10$	$\alpha(3) \rightarrow$ not possible
$S(4) = R = 4$	
$S(1) - P = 10$	// no Phantom Read.
$S(4) - R = 4$	



Now



Optimistic Concurrency Control

Begin txn:

	txn 1	txn 2
t ₁	id = 1, status = Free version = 1 (s)	id = 1, status = free version = 1 (s)
t ₂		id = 1, status = Book (w) version = 2
t ₃		Commit
t ₄	id = 1, status = Book (x) version = 2	

- Commit failed (version changed)
- in middle of txn.
- validation failed.
- no chances of deadlock.

Pessimistic Concurrency Control

- 1) T₁ will complete
- 2) T₂ will complete.

begin txn

t ₁	t ₂
Read(A)	Read(A)
Update(B)	→ no Read(B) update(B)
Commit(B)	•
	Update(B)
	Commit(B)

end txn

Problem

- 1) Deadlock.

Ex

T ₁	T ₂
Read(A)	Read(B)
Write(B)	Write(A)